Lab 6 实验报告

- **姓名**：徐培森
- **学号**：25300440003
- **邮箱**：25300440003@m.fudan.edu.cn

一、预估修改和实际修改对比

1.1 预估修改内容

在开发 Lab 6 之前，对需要修改的范围做了以下预估：

预估项	计划工作量
新增 JavaFX GUI 渲染层	大
复用 Lab 5 核心业务逻辑	不需要修改
新增点击事件处理	中
新增游戏专用渲染器	3~4 个独立文件
重构输入层	中
Demo 机制适配 GUI	小

1.2 实际修改内容

文件 / 变更	实际产出	与预估偏差
`JavaFXAPP.java`	JavaFX 主窗口，三栏布局 + 底部控制栏	符合预期
`GamePlatformController.java`	控制层，保持 MVC 分层	比预估更精简
`MinesweeperRenderer.java`	扫雷独立渲染器，支持颜色/插旗/踩雷效果	符合预期
`ChessRenderer.java`	国际象棋渲染器，实现两步点击 + 高亮选中	超出预期
`OthelloStyleRenderer.java`	黑白棋/和平棋渲染器，棋盘绿底 + 提示高亮	符合预期
`DefaultGamePanelRenderer.java`	备用渲染器，纯文本显示	符合预期
`GamePanelRenderer.java`	渲染器接口 + `coordToCmd` 工具方法	符合预期
`CellClickContext.java`	点击事件上下文，联系 Cell 和 Controller	新增未预估
`BaseGame.java` 新增方法	`onCellClick()`、`supportsPass()`、`supportsFlag()`、`createRenderer()`	略超预估

文件 / 变更	实际产出	与预估偏差
Lab 5 核心逻辑	Chess/MineField/Othello 代码零修改	符合预期
Scanner 废除	完全替换为事件驱动	符合预期
Demo 按钮 + 命令	支持 GUI 按钮和命令输入两种方式触发	符合预期

1.3 代码可控性结论

AI 的修改基本上处于可控范围内。核心业务逻辑层（chess 规则、扫雷布雷、黑白棋翻转）完全没有改动，复用率达 100%。变更集中在新增的渲染层和事件层，且这些新增文件都遵循清晰的分层结构，没有出现逻辑混乱或侵入式修改。唯一超出预估的是 `CellClickContext` 的设计，但引入它是为了解耦点击事件传递路径，属于合理扩展。

二、增量设计点

2.1 渲染机制

Lab 5：`TerminalUI.draw()` 中通过 `TextGraphics.putString()` 逐字符拼接棋盘，每次刷新全量重绘。颜色控制有限，棋子用 ASCII 字母（K/Q/R/B/N/P）表示。

Lab 6：每个游戏有自己的 `GamePanelRenderer` 实现，返回 JavaFX `StackPane` 控件节点。支持渐变色、Unicode 棋子符号、鼠标悬停高亮、选中高亮等丰富的视觉效果。棋盘通过 `GridPane` 布局管理，每次刷新只重建 GridPane 的子节点。

2.2 事件驱动

Lab 5：`GameController.run()` 的主循环中调用 `TerminalUI.readKey()` 阻塞等待键盘输入，然后通过正则匹配解析命令字符串。

Lab 6：每个棋盘的 `StackPane` 控件注册 `setOnMouseClicked` 事件处理器，通过 `CellClickContext` 将点击坐标、按钮类型传递给 `GamePlatformController.handleCellClick()`。

2.3 交互模式

以国际象棋为例：

- **Lab 5**：输入 `m e2 e4` → 程序解析 → 校验 → 执行
- **Lab 6**：点击棋子 e2（高亮选中）→ 点击 e4 → 全程鼠标操作

扫雷从 `f a1` / `a1` 命令改为左键翻开、右键插旗，并支持 Flag 模式按钮。

2.4 按钮控件与状态管理

新增了底部控制栏的快捷按钮，在不同游戏模式选择显示：

按钮	用途
Pass	仅 reversi 无棋可下时使用
Flag: ON/OFF	扫雷插旗模式切换
Demo	一键启动全部游戏的自动演示
Quit	安全退出

2.5 游戏自主渲染能力

`BaseGame` 新增 `createRenderer()` 方法，每个游戏实例可返回自己的渲染器。渲染器内部维护选择状态（如 ChessRenderer 的 `selectedPosition`），做到渲染与业务逻辑彻底分离。

2.6 Demo 逻辑变化

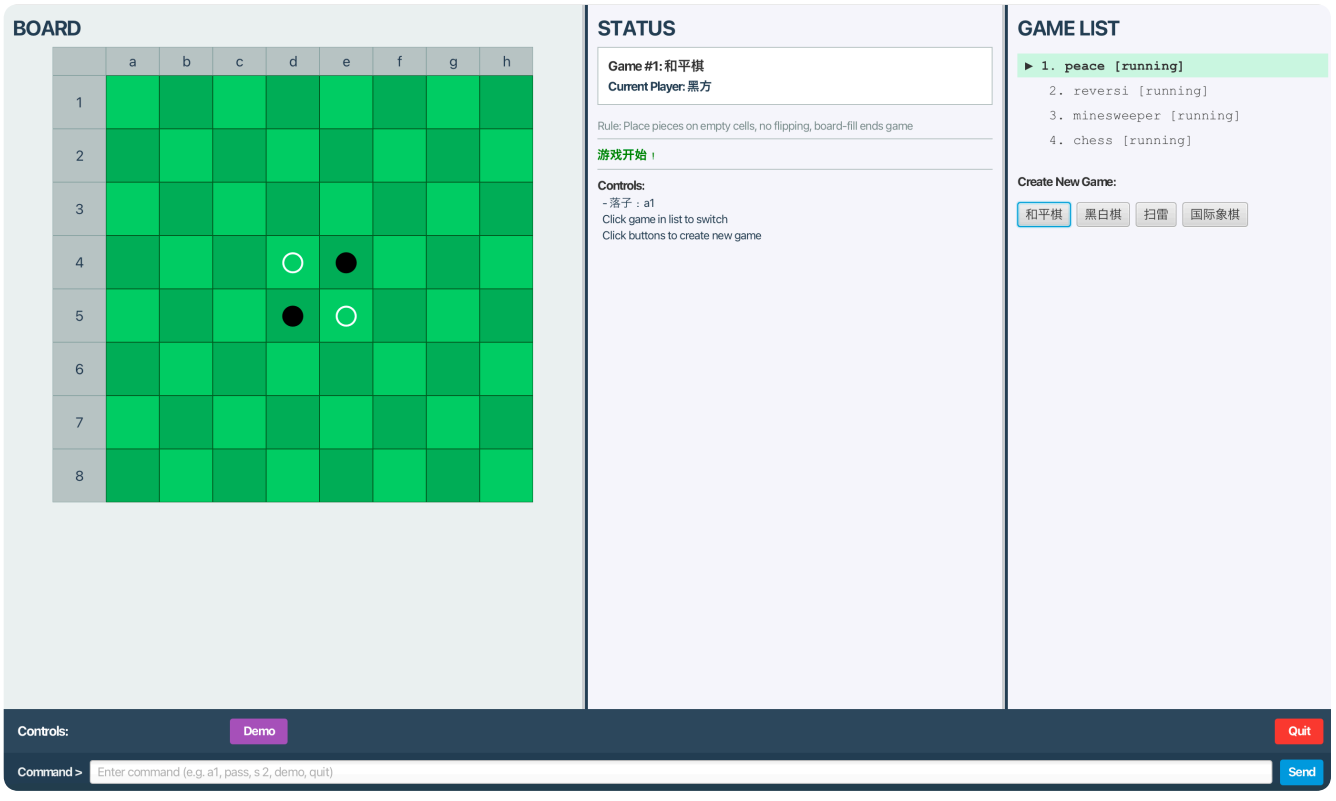
Lab 5 的 Demo 是在主循环中通过 `tickDemo()` 轮询执行的；Lab 6 则通过 JavaFX `Timeline` 动画机制实现，各游戏依次创建临时实例并播放预设步骤，结束后自动清理。

2.7 交互式游戏列表

Lab 5 的右侧游戏列表只是静态文本；Lab 6 中每个列表项是可点击的 `Label`，支持鼠标单击切换到对应对局。

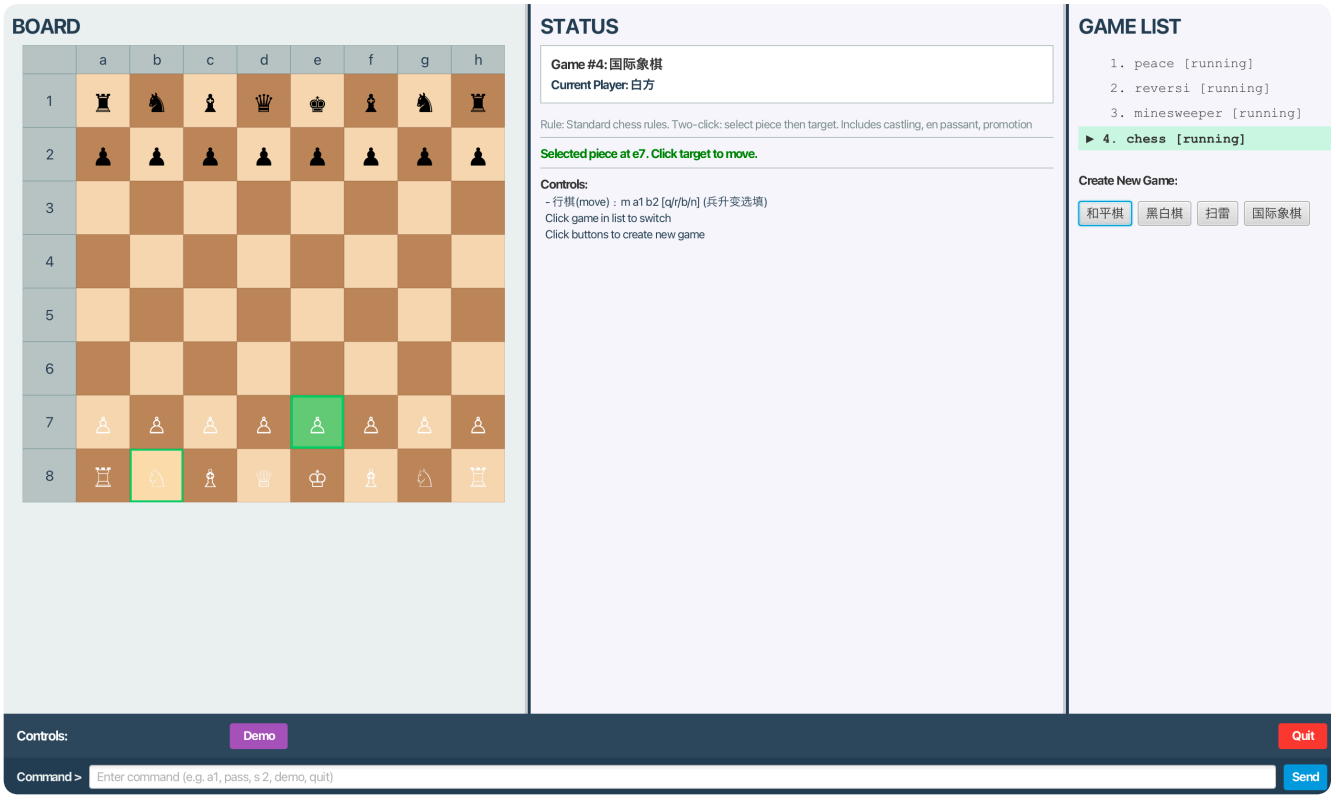
三、关键实验过程运行截图

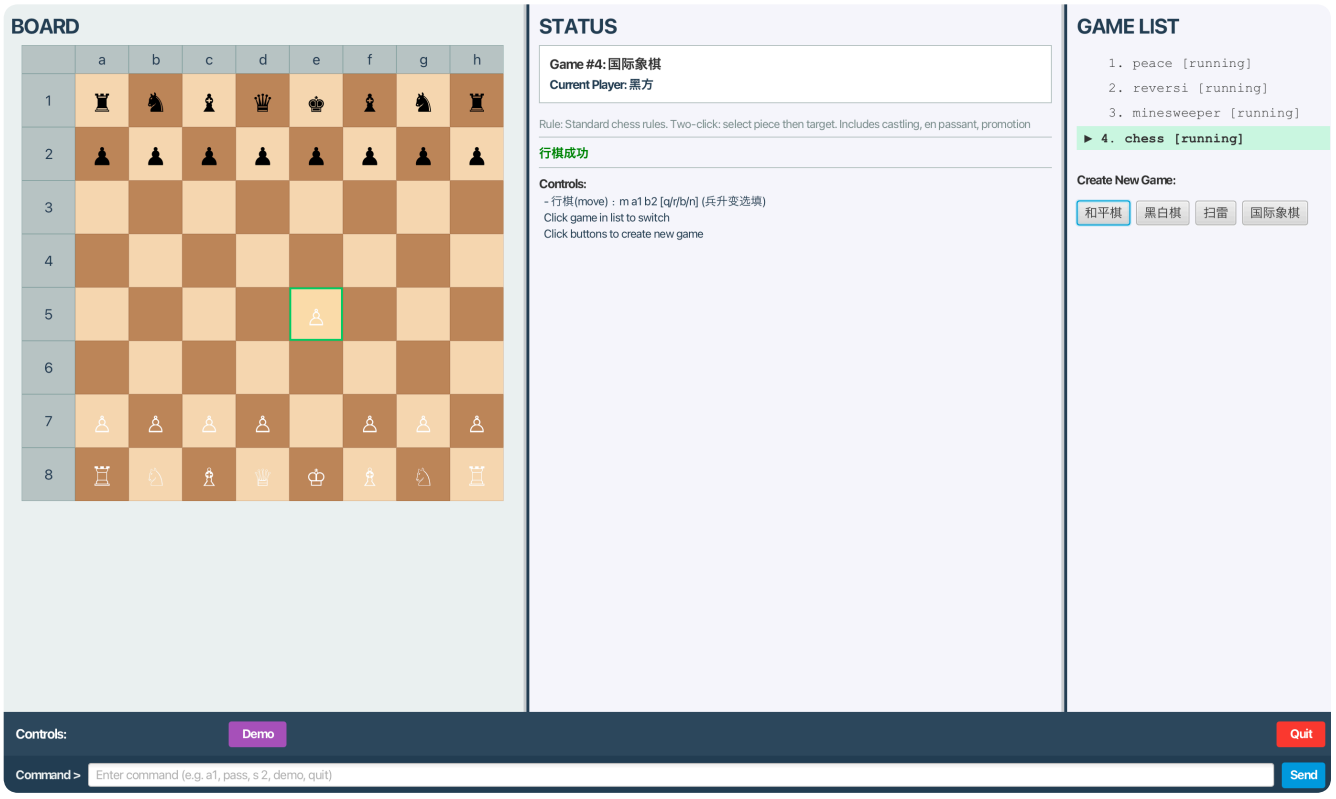
3.1 初始启动界面



- 启动后默认加载 4 局游戏（peace/reversi/minesweeper/chess），当前显示的是 peace 棋盘。状态面板显示当前游戏编号、回合信息、规则说明。右侧列表高亮当前对局。

3.2 国际象棋两步点击交互

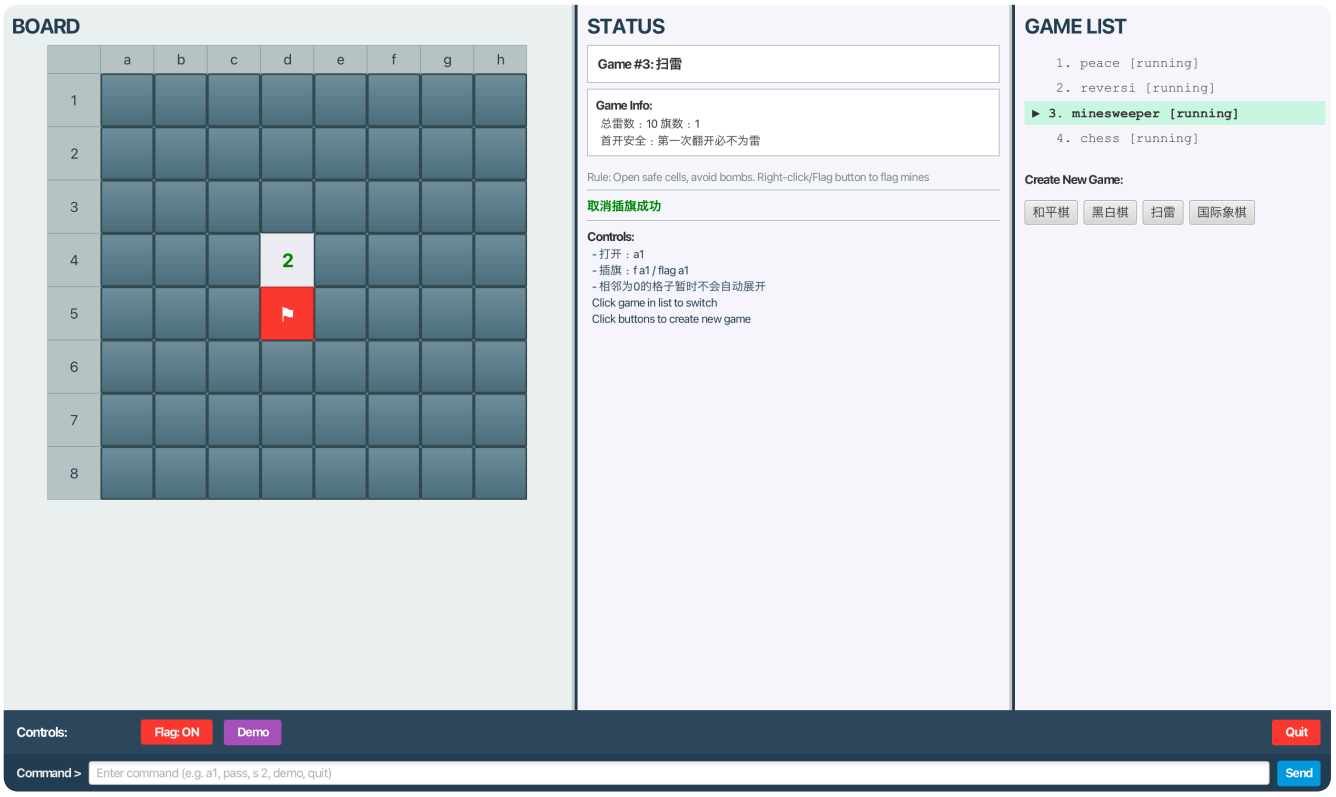


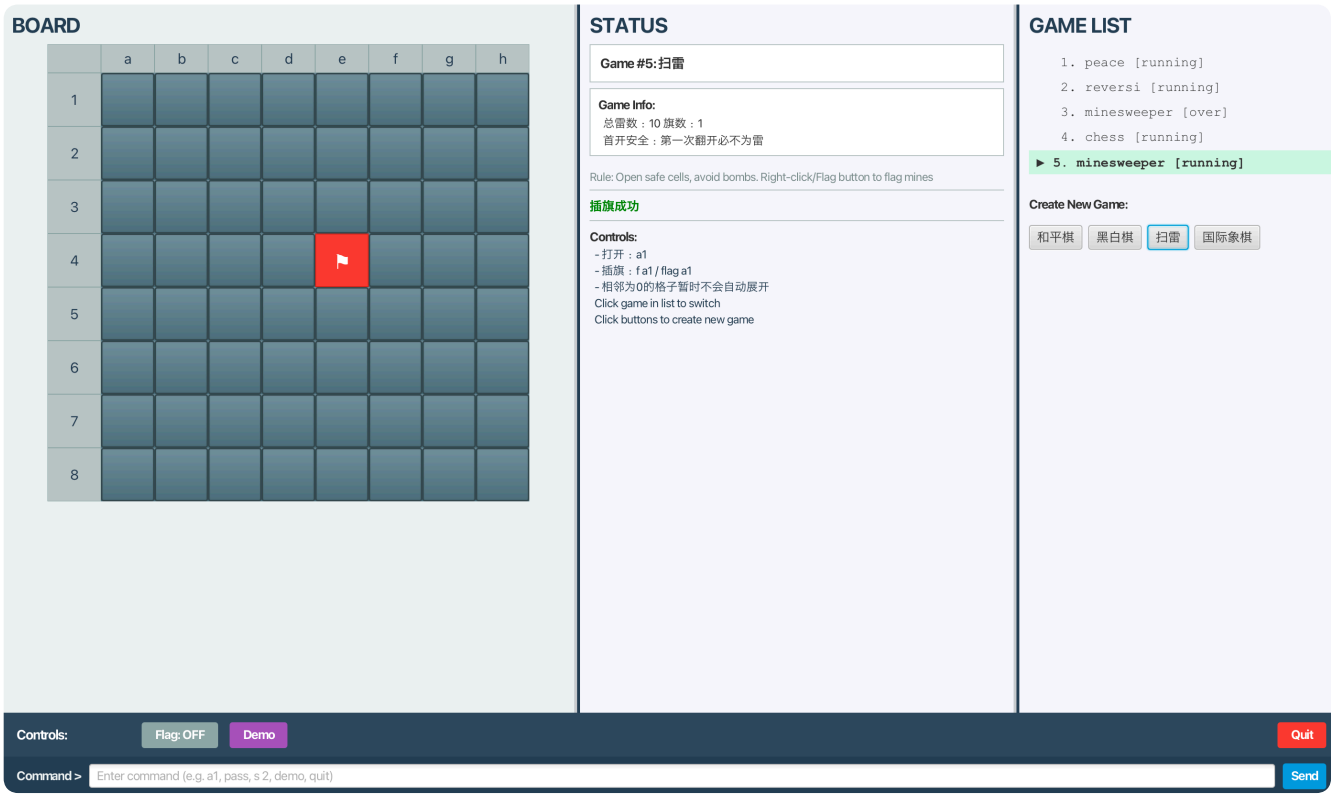


说明：

1. 点击白兵 e7 → 该格高亮（绿色边框），状态栏显示 "Selected piece at e7"
2. 点击 e5 → 兵走到 e5，回合切换到黑方

3.3 扫雷交互

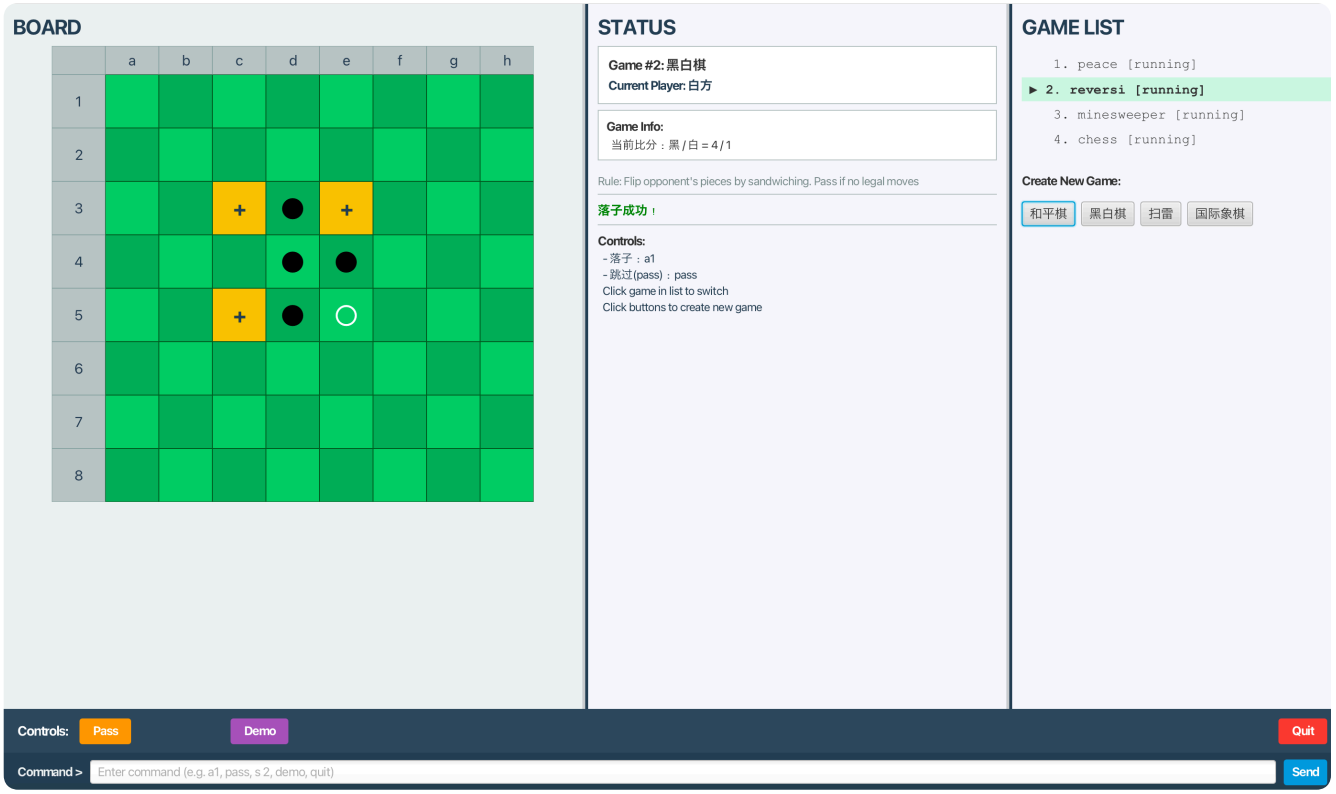




说明：

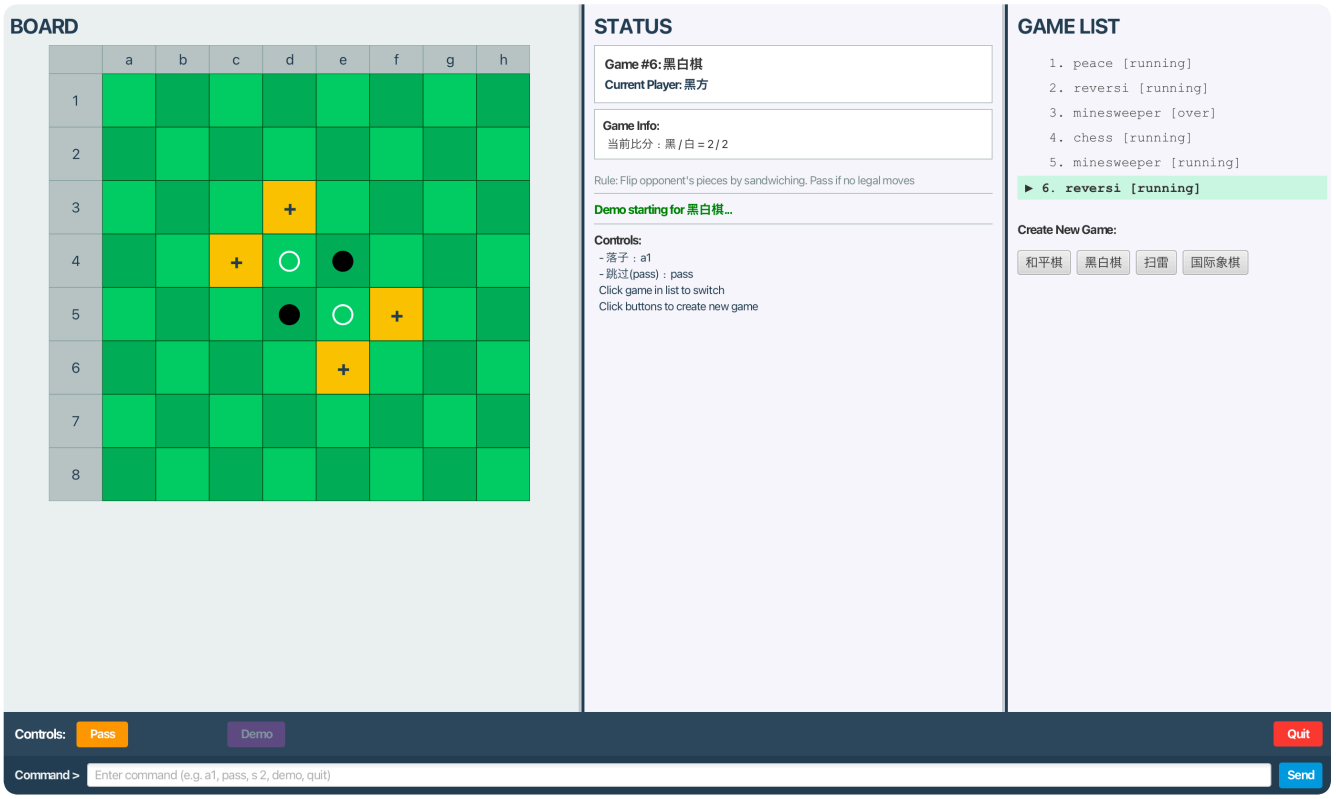
1. 单击插旗按钮，鼠标左键用于插旗。
2. 取消点击插旗按钮，鼠标右键用于插旗。

3.4 黑白棋合法位置提示



- reversi 模式下，可落子位置以黄色格子的 `+` 号高亮显示。

3.5 Demo 模式



点击 Demo 按钮后，程序依次创建临时游戏实例并运行预设走法，状态面板同步展示每一步的讲解文字。

四、TUI 与 GUI 架构比较

4.1 渲染机制对比

TUI（Lab 5）：基于 Lanterna 的字符渲染

Lab 5 的渲染全部发生在 `TerminalUI` 类中，本质是 "**按坐标往终端写字符**"。核心渲染流程如下：

```
TerminalUI.draw()
  │
  ├─ screen.clear()          ← 全量清屏
  ├─ 用 TextGraphics 逐格 putString(x, y, char)
  │    ├─ 棋盘点阵字符：K / Q / R / B / N / P
  │    ├─ 颜色控制：setForegroundColor(ANSI 16色)
  │    └─ 无像素级控制，只能操作字符网格
  ├─ 手动拼接状态文字
  └─ screen.refresh()        ← 刷新终端缓冲区
```

特征：

- **坐标模型**：所有元素的位置需要手动计算 `(x, y)` 字符坐标，布局代码与渲染逻辑紧耦合。例如要画一个 8×8 棋盘，你需要手算每个格子从第几行第几列开始画。
- **全量刷新**：每次状态变化都需要 `screen.clear()` + 全量重绘。即使只动了一个格子，整个棋盘也要重画一遍。
- **颜色限制**：只能使用默认颜色，无法表达渐变色、阴影或半透明。

GUI（Lab 6）：基于结点的图形渲染

Lab 6 的渲染采用 **JavaFX 场景图**，每个 UI 元素是场景图中的一个节点。核心渲染流程：

```
JavaFXAPP.refreshBoard()
  │
  ├─ boardGrid.getChildren().clear()   ← 清空 GridPane 子节点
  ├─ for each cell (r, c):
  │    └─ renderer.renderCell(game, r, c, isHint, ctx)
  │         └─ 返回 StackPane (内含 Text, 样式, 事件绑定)
  │              ├─ 背景色：-fx-background-color: #F0D9B5
  │              ├─ 棋子：Text(♔♕♖♗♘♙), Font(32), fill(BLACK/WHITE)
  │              ├─ 事件：setOnMouseClicked / setOnMouseEntered /
setOnMouseExited
  │              └─ 布局：StackPane 自动居中
  └─ boardGrid.add(cell, c+1, r+1)     ← GridPane 自动布局
```

技术特征：

- **硬件加速**：JavaFX 支持硬件加速。场景图变更会被合成为绘制指令，由 GPU 完成运算。
- **场景图模型**：UI 以树状结构组织，每个节点（StackPane、Text、GridPane）都维护自己的位置、大小、样式，由 JavaFX 布局引擎自动计算。

- **增量更新**：场景图只重建 `GridPane` 的子节点，整个页面不需要全量刷新。JavaFX 内部只会重绘变化的部分。
- **颜色无限制**：RGB 任意，支持线性渐变色。

1.2 UI 设计模式对比

Lab 5：命令式 UI 构建

Lab 5 的界面直接调用 Lanterna API 在终端上"画"出界。

```
graphics.putString(2, 1, "GAME PLATFORM");
graphics.putString(2, 3, "1. Peace");
graphics.putString(2, 4, "2. Reversi");
graphics.putString(2, 5, "3. Minesweeper");
graphics.putString(2, 6, "4. Chess");

// 画棋盘：每个格子位置手动计算
for (int r = 0; r < 8; r++) {
    for (int c = 0; c < 8; c++) {
        int x = c * 4 + 10;
        int y = r * 2 + 3;
        graphics.putString(x, y, pieceChar);
    }
}
```

问题：

- 布局逻辑与渲染逻辑完全混在一起
- 坐标需要手算，改布局要改所有坐标
- 无组件化，无法复用 UI 片段
- 没有 CSS / 样式分离的概念

Lab 6：组件式 UI

Lab 6 使用 **JavaFX 组件** 构建界面，布局由容器类自动管理：

```
BorderPane root = new BorderPane();

// 三栏布局
HBox mainContent = new HBox();
mainContent.getChildren().addAll(boardPanel, sep1, statusPanel, sep2,
gameListPanel);

// 权重分配
HBox.setHgrow(boardPanel, Priority.ALWAYS);    // 棋盘区域优先扩展
HBox.setHgrow(statusPanel, Priority.ALWAYS);
HBox.setHgrow(gameListPanel, Priority.ALWAYS);
```

```
11
12  root.setCenter(mainContent);
13  root.setBottom(bottomArea);    // 底部控制栏
```

优势：

- **布局容器自动计算**：BorderPane 自动分"上中下"，HBox 自动排"左中右"，GridPane 自动排"行列格"。程序员只需要声明"什么往哪放"，不需要关心具体像素。
- **组件树**：Panel（容器）→ Label（文字）→ Button（按钮）→ StackPane（格子），每一层都是一个可独立管理、替换、复用的组件。

本项目中使用的组件层级如下：

```
BorderPane
  ├─ Center: HBox (三栏)
  │    ├─ boardPanel (VBox)
  │    │    └─ GridPane (8×8 棋盘)
  │    ├─ statusPanel (VBox)
  │    │    ├─ Label "STATUS"
  │    │    ├─ infoCard (VBox)
  │    │    ├─ msgBox
  │    │    └─ cmdBox
  │    └─ gameListPanel (VBox)
  │         └─ gameListBox (VBox)
  │              └─ Label (每局游戏)
  └─ Bottom: VBox
       ├─ HBox (操作按钮)
       └─ HBox (命令输入)
```

1.3 事件处理模型对比

TUI：阻塞式轮询

Lab 5 的事件处理基于 **单线程阻塞循环**：

```
GameController.run()
  │
  ├─ while (!quit) {
  │    ├─ if (demoMode) → tickDemo()        ← 总是等待 demo 状态
  │    ├─ else → TerminalUI.readKey()       ← 阻塞等待键盘
  │    │          └─ Lanterna KeyStroke 读取
  │    ├─ handleEnter(input)                ← 解析命令
  │    └─ TerminalUI.draw()                 ← 刷新显示
  │    }
  └─ screen.stop()
```

特征：

- **同步阻塞**：`readKey()` 会阻塞线程直到用户按下按键，期间什么事都干不了。
- **单线程**：所有游戏逻辑、渲染、输入处理都在同一条线程上顺序执行。
- **Demo 与用户输入冲突**：Demo 播放时只能通过 `pollInput()` 非阻塞地轮询检查用户是否按了退出键，无法同时监听用户操作。
- **无鼠标事件**：终端界面不支持鼠标点击，所有交互必须通过键盘。

GUI：事件驱动

Lab 6 的事件处理基于 **事件队列 + 监听器注册**：

```
JavaFX Application Thread
  │
  ├─ [事件源] 用户点击棋盘格子
  │    └─ setOnMouseClicked(e → ...)  ← 注册的回调被调用
  │         └─ ctx.handleCellClick(row, col, isSecondary)
  │              └─ controller.handleCellClick()
  │                   └─ game.onCellClick()
  │                        └─ 更新游戏状态
  │              └─ refreshAll()  ← 触发 UI 更新
  │
  ├─ [事件源] 用户按下 Enter
  │    └─ setOnKeyPressed(e → if ENTER → processCommandInput())
  │         └─ dispatchCommand(input)
  │         └─ 根据 CommandResult 执行对应动作
  │
  ├─ [事件源] 用户点击 Demo 按钮
  │    └─ setOnAction(e → handleDemo())
  │         └─ new Timeline() → 定时触发 KeyFrame
  │
  └─ [事件队列管理] JavaFX 内部管理事件循环
```

关键特征：

- **异步非阻塞**：事件处理器注册后立即返回，不阻塞 UI 线程。
- **多来源事件共存**：鼠标点击、键盘输入、按钮点击可以同时监听，互不干扰。
- **事件与渲染解耦**：事件处理器只修改模型状态，UI 通过 `refreshAll()` 主动同步。渲染不需要在事件循环中轮询。
- **Timeline 动画**：Demo 通过 `Timeline` + `KeyFrame` 实现，每一步在指定延迟后自动触发，不影响用户同时操作其他按钮。

```
cell.setOnMouseClicked(e -> {
    // 左键/右键区分
    ctx.handleCellClick(row, col, e.getButton() == MouseButton.SECONDARY);
});

```

```
// Demo 使用事件驱动而非轮询
Timeline timeline = new Timeline();
timeline.getKeyFrames().add(new KeyFrame(
    Duration.millis(delay * i),          // "在 delay*i 毫秒后执行"
    e -> game.processCommand(step.command())
));
timeline.play();                          // 启动计时器，不阻塞
```

Lab 6 中一个重要的解耦设计是 `CellClickContext`。它充当了 **事件生产者和处理者之间的桥梁**:

```
// 创建 CellClickContext 时传入所有回调
CellClickContext ctx = new CellClickContext(
    this.refreshAll,                                    // 刷新整个UI
    this.refreshBoard,                                  // 刷新棋盘
    this.refreshStatus,                                 // 刷新状态栏
    (msg, err) -> { controller.setStatus(msg, err); },  // 设置状态
    flagToggleButton::isSelected,                       // Flag 模式状态
    (row, col, isSec, isFlag) -> {                      // 点击处理
        controller.handleCellClick(row, col, isSec, isFlag);
        refreshAll();
    }
);

// 渲染器只需调用:
ctx.handleCellClick(row, col, isSecondary);
// 不需要知道背后是谁在处理、怎么处理
```

五、代码复用与解耦策略分析

5.1 总体复用原则

Lab 6 的设计目标之一是最大化复用 Lab 5 的业务逻辑代码。实现这一目标依赖两个关键策略:

1. **关注点分离**：将"游戏规则"与"展示方式"彻底剥离。游戏逻辑只处理数据结构和规则判定，不依赖任何 UI 框架。
2. **接口抽象**：使用接口（`GamePlugin`、`GamePanelRenderer`、`HintProvider`）定义扩展点，让新增功能和旧有代码通过接口交互，而不是直接修改旧代码。

5.2 完全复用模块

5.2.1 核心游戏规则层

文件	功能	为什么不用改
`ChessBoard.java`	棋盘数据结构（8×8 数组 + set/get/边界检查）	纯数据容器，不依赖 UI
`ChessPiece.java`	棋子枚举（EMPTY/WKING/BKING/WPAWN/...）	纯常量定义
`ChessMoveValidator.java`	走法生成 + 将军/将杀判定	纯逻辑计算，输入输出都是数据对象
`ChessMoveCandidate.java`	候选走法数据类	纯数据结构
`ChessMove.java`	已执行的走法记录	纯数据结构
`Board.java`（Othello）	黑白棋棋盘 + 翻转算法	纯数据操作
`Piece.java`（Othello）	黑白棋子枚举	纯常量定义
`MineField.java`	布雷 + 计数 + 展开算法	纯状态机，输入输出是 CellStatus 枚举
`CellStatus.java`	扫雷格子状态枚举	纯常量定义

这些文件不改的原因：它们实现的是规则而非交互。例如规则类只需要知道棋子在哪、要往哪走，它不关心这个信息是来自键盘输入还是鼠标点击。

2.2.2 游戏文案层

文件	功能
`ChessTexts.java`	国际象棋文案（走法说明、提示信息）
`MinesweeperTexts.java`	扫雷文案
`OthelloTexts.java`	黑白棋文案

2.2.3 基础模型层

文件	功能	说明
`Position.java`	坐标数据类 `(row, col)`	Lab 5 和 Lab 6 共用同一个 `record`
`TurnBasedGame.java`	回合制游戏接口	接口仍可继续使用
`HintProvider.java`	提示接口 `getLegalMoves()`	完全复用

2.2.4 插件系统层

文件	功能	说明
`GamePlugin.java`	插件接口	接口定义没动，Lab 6 新增功能在 BaseGame 上扩展

文件	功能	说明
`GamePluginRegistry.java`	插件注册表	SPI 注册 + 查询，完全复用
`ChessPlugin.java`	象棋插件	唯一变化是 `isDefaultGame()` 返回 `true`（Lab 5 也是这么写的）
`MinesweeperPlugin.java`	扫雷插件	同上
`PeacePlugin.java`	和平棋插件	同上
`ReversiPlugin.java`	黑白棋插件	同上

2.2.5 多局管理层

文件	功能	复用方式
`MultiBoardManager.java`	多游戏实例管理	完全复用，Lab 6 新增了 `defaultsLoaded` 懒加载（加在既有逻辑之上，不改原有方法）

2.3 重构与解耦的部分

2.3.1 BaseGame 基类 — 扩展

`BaseGame` 是复用策略的核心 —— **Lab 5 的所有抽象方法一字不改，Lab 6 新增的方法全部作为扩展点添加**：

```
// ===== Lab 5 既有（未动） =====
public abstract boolean processCommand(String cmd);
public abstract List<String> getExtraStatusLines();
public abstract List<String> getAvailableCommands();
public abstract String getType();
public abstract String getDisplayName();
public abstract String getCellChar(int row, int col);

// ===== Lab 6 新增（不破坏旧接口） =====
public void onCellClick(int row, int col, boolean isSecondary, boolean
isFlagMode) {}
    // 鼠标点击 → 游戏逻辑的入口，子类可覆写

public boolean supportsPass() { return false; }
    // 是否需要 Pass 按钮（Reversi 子类覆写为 true）

public boolean supportsFlag() { return false; }
    // 是否需要 Flag 按钮（Minesweeper 子类覆写为 true）

public void prepareDemo() {}
    // Demo 前置初始化（Minesweeper 固定布雷）
```

```
21
22  public GamePanelRenderer createRenderer() {
23      return new DefaultGamePanelRenderer();
24  }
25      // 返回自定义渲染器，不覆写则用默认实现
```

这种设计的好处：

- 旧子类无需任何改动
- 新功能以"可选覆写"方式提供，子类只覆写自己需要的
- 默认行为是"不做任何 GUI 特有的事"，保证非 GUI 环境下也能正常工作

2.3.2 输入层 — 完全替换

Lab 5	Lab 6	重构策略
`TerminalUI.readKey()` 阻塞等待	`setOnMouseClicked` 事件回调	全换
`GameController.handleEnter()` 解析命令字符串	`GamePlatformController.dispatchCommand()`	保留命令解析逻辑，去掉 Scanner 依赖
不支持鼠标	每个格子注册独立事件监听	全新增

2.3.3 视图层 — 完全替换

Lab 5	Lab 6	重构策略
`TerminalUI.draw()` 全量字符绘制	`JavaFXAPP.refreshBoard()` + 各 Renderer	全换
无渲染器概念	`GamePanelRenderer` 接口 + 4 个实现	全新增

视图层替换的核心思路是 **引入视图接口**，将渲染这件事从游戏逻辑中完全抽离：

```
1   public interface GamePanelRenderer {
2       // 画一个格子，返回 JavaFX StackPane 控件
3       StackPane renderCell(BaseGame game, int row, int col,
4                            boolean isHint, CellClickContext ctx);
5
6       void clearSelection();
7           // 清除选中状态（ChessRenderer 清空 selectedPosition）
8
9       String getRuleText();
10          // 该游戏的规则说明文字
11
12      static String coordToCmd(int row, int col) {
13          return (char) ('a' + col) + String.valueOf(row + 1);
14          // 坐标转换工具：(row=0, col=0) → "a1"
```

```
        }
    }
```

渲染器实现对比：

渲染器	特点	视觉
`ChessRenderer`	棕白交替棋盘，选中高亮（绿色），悬停变色	Unicode 棋子 ♔♕♖♗♘♙
`MinesweeperRenderer`	渐变色未翻格，红色雷区，踩雷黄色高亮，数字着色	不同雷数的颜色不同
`OthelloStyleRenderer`	绿色棋盘，可落子位置黄色 `+` 号提示	黑子 ● 白子 ○
`DefaultGamePanelRenderer`	灰白交替兜底，纯文字	无特效

2.3.4 Demo 机制 — 重写

Lab 5	Lab 6
`GameController.tickDemo()` 在主循环中轮询执行	`JavaFXAPP.playNextDemo()` 使用 Timeline 动画
Demo 时阻塞用户输入	Demo 时按钮仍可点击
所有 Demo 步骤在单次 tick 中串行推进	每个 KeyFrame 在指定延迟后自动触发
`getDemoSteps()` 包含副作用（生成固定雷场）	`prepareDemo()` 分离初始化逻辑

Demo 模块的重写原因：

1. **执行机制不同**：TUI 的主循环可以自然地在每轮循环中推进 Demo 一步；GUI 则需要在事件循环中设置定时器触发每一步。
2. **线程模型不同**：TUI 的 Demo 是同步的（在主循环里一步步走），GUI 的 Demo 是异步的。

2.4 解耦的关键设计模式

2.4.1 策略模式 — GamePanelRenderer

每个游戏有自己独有的渲染行为，通过 `GamePanelRenderer` 接口定义策略，`BaseGame.createRenderer()` 选择具体策略：

```
// 策略接口
public interface GamePanelRenderer {
    StackPane renderCell(...);
    String getRuleText();
}

// 具体策略 A
public class ChessRenderer implements GamePanelRenderer { ... }
```

```
// 具体策略 B
public class MinesweeperRenderer implements GamePanelRenderer { ... }
// 具体策略 C
public class OthelloStyleRenderer implements GamePanelRenderer { ... }
```

2.4.2 工厂模式 — BaseGame.createRenderer()

每个游戏子类可以决定自己使用哪个渲染器：

```
// BaseGame 定义工厂方法
public GamePanelRenderer createRenderer() {
    return new DefaultGamePanelRenderer();  // 默认实现
}

// ChessGame 覆写
@Override
public GamePanelRenderer createRenderer() {
    return new ChessRenderer();  // 返回专用渲染器
}
```

六、总结

Lab 6 在 Lab 5 的完整业务逻辑基础上，成功将终端界面升级为 JavaFX 图形界面。整个过程中：

1. **代码可控性良好**：AI 的修改集中在新增的渲染层和事件层，核心业务逻辑零修改，没有出现不可控的侵入式变更。
2. **架构解耦清晰**：通过 `GamePanelRenderer` 接口和 `CellClickContext` 桥接层，做到了渲染与业务、事件与处理的完全分离。
3. **插件化架构成熟**：新增游戏只需实现统一接口即可接入大厅，主程序无需修改，具备高可扩展性。
4. **增量设计合理**：所有新增功能都是对既有框架的自然扩展，没有引入架构冲突。